

Istanbul Medeniyet University
Department of Computer Engineering

BIL 250 Operating Systems — Spring 2026
Final Examination

İsim ve Soyisim : _____

Öğrenci No : _____

İmza : _____

Her kısım 20 puandır. Kısımlardaki her bir boşluk/cevap eşit puan olarak değerlendirilecektir.

Question	Points	Score
1	20	
2	20	
3	20	
4	20	
5	20	
Total:	100	

Part I: Processes, Threads & Advanced Synchronization

20

1. (a) Aşağıdaki kod bloğu çalıştırıldığında toplamda kaç defa "X" yazdırılır? (*English: When the following code segment is executed, how many total times will the character "X" be printed?*)

```
fork();  
if (fork() == 0) {  
    fork();  
}  
printf("X\n");
```

Cevap (Answer): _____ **6** _____

Solution: 6 tane "X" yazdırılır. / 6 times.

- (b) Aşağıdaki ifadelerin yanına **True** veya **False** yazınız. (*English: Write **True** or **False** next to the statements.*)

- i. Linux çekirdeğinde, `clone()` sistem çağrısı ile oluşturulan iş parçacıkları, ana süreç ile hiçbir bellek adresini paylaşmayan tamamen izole Sayfa Tablolarına sahiptir. (*English: In the Linux kernel, threads created via the `clone()` system call have completely isolated Page Tables that do not share any memory addresses with the parent.*)

Cevap (Answer): _____ **False** _____

- ii. `pthread_cond_wait()` çağrısını bir `while` döngüsü yerine `if` ifadesi ile sarmak güvenlidir, çünkü hatalı uyandırmalar (spurious wakeups) donanım tarafından otomatik olarak işlenir. (*English: Wrapping the `pthread_cond_wait()` call inside an `if` statement instead of a `while` loop is safe because spurious wakeups are handled automatically by the hardware.*)

Cevap (Answer): _____ **False** _____

- iii. Donanım kesmelerini devre dışı bırakmak, modern çok çekirdekli CPU'larda karşılıklı dışlama (mutual exclusion) sağlamak için yetersizdir, çünkü bu işlem yalnızca komutu çalıştıran yerel çekirdekteki kesmeleri devre dışı bırakır. (*English: Disabling hardware interrupts is insufficient for achieving mutual exclusion on modern multi-core CPUs because it only disables interrupts on the local core running the instruction.*)

Cevap (Answer): _____ **True** _____

- (c) POSIX standartlarına göre çalışan bir sayma semaforunun (counting semaphore) başlangıç değeri **2**'dir. Bu semafor üzerinde sırasıyla şu işlemler gerçekleştiriliyor (Semafor sayacının negatif değerlere düşebildiğini varsayınız): (*English: A POSIX counting semaphore is initialized to 2. The following sequence of functions is executed sequentially on it (Assume the internal*

semaphore counter drops into negative values to track blocked threads):

`sem_wait()`, `sem_wait()`, `sem_post()`, `sem_wait()`, `sem_wait()`, `sem_post()`

Bu işlemler dizisi tamamlandıktan sonra semaforun son sayısal değeri kaç olur? (*What is the final numerical value of the semaphore counter after all operations complete?*)

Answer (Cevap): _____ **0** _____

Solution: Başlangıç = 2. (Initial = 2)

1. `sem_wait()` → counter = 1

2. `sem_wait()` → counter = 0

3. `sem_post()` → counter = 1

4. `sem_wait()` → counter = 0

5. `sem_wait()` → counter = -1 (çağırان iş parçacığı bloke olur / the calling thread blocks)

6. `sem_post()` → counter = 0 (bloke olmuş bir iş parçacığı uyanır / a blocked thread is woken up)

Son değer 0'dır. (Final value is 0.)

(d) Thread A ve Thread B bir kritik bölge kilidi için yarışmaktadır. **Spinlock** ve **Mutex** davranış farklarını göz önüne alarak boşlukları doldurunuz. (*English: Thread A and Thread B compete for a critical section lock. Fill in the blanks describing the behavior of **Spinlock** vs. **Mutex**.*)

(a) Eğer Thread A bir (*English: If Thread A holds a, ..*) _____ **Spinlock** _____ tutuyorsa, Thread B, kilit serbest bırakılana kadar CPU çekirdeğinde meşgul bekleme (busy-waiting) döngüsünde döner. (*..., Thread B will spin in a busy-waiting loop on the CPU core consuming cycles until the lock is released.*)

(b) Eğer Thread A bir (*English: If Thread A holds a,...*) _____ **Mutex** _____ tutuyorsa, Thread B zamanlayıcı tarafından uyutulur ve başka hazır süreçlerin çalışmasına izin vermek için bir bağlam değişimi (context switch) tetiklenir. (*..., Thread B will be put to sleep by the scheduler, triggering a context switch to allow other ready processes to execute.*)

(c) Threadlerin kullanıcı seviyesinde veya kernel seviyesinde planlanması ilgili aşağıdaki ifadelerin hangi tür threadleri tanımladığını **user-level** veya **kernel-level** olarak belirtiniz. *English: For each statement below, indicate whether it describes **user-level threads** or **kernel-level threads**.*

1. _____ **user-level** _____ Yönetimi kullanıcı alanı kütüphanesi tarafından yapılır. / Management is done by a user-space library.

2. _____ **kernel-level** _____ İdaresi çekirdek tarafından yapılır. / Management is done by the kernel.

3. _____ **user-level** _____ Bloklayan bir sistem çağrısı tüm iş parçacıklarını bloke eder. / A blocking system call blocks all threads.

4. _____ **kernel-level** _____ Bloklayan bir sistem çağrısı sadece çağırان iş parçacığını bloke eder. / A blocking system call blocks only the calling thread.

5. _____ **user-level** _____ Çoklu çekirdeklerden tam olarak yararlanamaz. / Cannot fully utilize multiple cores.

6. _____ **kernel-level** _____ Çoklu çekirdeklerden yararlanabilir. / Can utilize multiple cores.

7. _____ **user-level** _____ Bağlam değiştirme maliyeti düşüktür (fonksiyon çağrısı). / Context switch cost is low (function call).

8. _____ **kernel-level** _____ Bağlam değiştirme maliyeti yüksektir (çekirdek giriş/çıkışı). / Context switch cost is high (kernel entry/exit).

(d) Aşağıdaki multithreading model tanımlarını uygun model adlarıyla eşleştiriniz. Modeller: **Many-to-one**, **One-to-one**, **Many-to-many**

English: Match the multithreading model descriptions with the correct model names.

1. Many-to-one: Tüm kullanıcı iş parçacıkları tek bir çekirdek iş parçacığına eşlenir. *many user threads associated with a single kernel thread.*
 2. One-to-one: Her kullanıcı iş parçacığı bir çekirdek iş parçacığına eşlenir. *every user thread associated with a separate kernel thread.*
 3. Many-to-many: Çok sayıda kullanıcı iş parçacığı daha az sayıda çekirdek iş parçacığına eşlenir. *User threads associated with a number of kernel threads.*
- (e) Aşağıda kodda 10 adet thread eşzamanlı olarak ‘increment_counter’ fonksiyonunu çalıştırmaktadır. (English: 10 concurrent threads invoke the ‘increment_counter’ function given below.)

```
int global_counter = 0; //Paylaşılan global degisken / Shared global variable
void* increment_counter(void* arg) {
    for (int i = 0; i < 1000; i++) {
        global_counter++;
    }
    return NULL;
}
```

- i. Yarış durumuna (Race condition) sebep olan **Kritik Bölge (Critical Section)** satırını/ifadesini yazınız. (English: Identify the exact expression or line that constitutes the **Critical Section** and causes a race condition in the code above.)

Answer (Cevap): global_counter++;

Solution: global_counter++; kritik bölgedir çünkü paylaşılan durum üzerinde senkronize edilmemiş oku-değiştir-yaz işlemleri yapar. / It is the critical section because it performs unsynchronized read-modify-write operations on shared state.

- ii. ‘global_counter++’ işleminin düşük seviyeli donanım/assembly adımları ile ilgili ifadelerin yanına **True** veya **False** yazınız. (English: Write **True** or **False** next to the statements regarding the low-level assembly execution steps of the ‘global_counter++’ operation.)
 - α) CPU, ‘global_counter++’ satırını donanım seviyesinde tek ve atomik bir komut olarak yürütür, böylece çok çekirdekli saat ara vermesini (interleaving) engeller. (English: The CPU executes the line ‘global_counter++’ as a single, atomic instruction at the hardware level, preventing multi-core clock interleaving.)
Cevap (Answer): False
 - β) İlk fiziksel adım, ‘global_counter’ın mevcut değerini ana bellekten veya önbellekten bir CPU yazmacına getirmeyi/yüklemeyi içerir. (English: The first physical step involves fetching/loading the current value of ‘global_counter’ from main memory or cache into a CPU register.)
Cevap (Answer): True
 - γ) Bir iş parçacığı kendi yerel yazmacını artırdıktan sonra ancak değeri belleğe geri yazmadan önce bir bağlam değişimi (context switch) olursa yarış durumu (race condition) ortaya çıkar. (English: A race condition manifests if a context switch breaks the execution order after a thread increments its local register but before it writes the value back to memory.)
Cevap (Answer): True

Part II: CPU Scheduling Algorithms

20

2. (a) Aşağıda bir sisteme aynı anda (bütün geliş zamanları $T = 0$) gelen süreçler ve çalışma süreleri (burst time) verilmiştir: (English: Consider the following set of processes arriving at time $T = 0$ with the given burst times.)

Process (Süreç)	Burst Time (ms)
P1	10
P2	4
P3	2

- (a) **Shortest Job First (SJF)** algoritmasına göre ortalama bekleme süresini hesaplayınız. (*English: Calculate the average waiting time using the **Shortest Job First (SJF)** scheduling algorithm.*)

Answer (Cevap): _____ **2.66** _____ ms

Solution: Sıra (Order): P3 (2ms) → P2 (4ms) → P1 (10ms). Bekleme süreleri (Waiting times): P3=0, P2=2, P1=6. Ortalama (Average) = $(0+2+6)/3 = 2.66$ ms.

- (b) Kuantum süresi $q = 3$ ms olan **Round Robin (RR)** algoritmasına göre P1 sürecinin toplam bekleme süresini hesaplayınız. (*English: Calculate the total waiting time for process **P1** under **Round Robin (RR)** scheduling with a time quantum of $q = 3$ ms.*)

Answer (Cevap): _____ **6** _____ ms

Solution: P1, 3'ten 8'e (5 ms) ve 11'den 12'ye (1 ms) bekler. Toplam = 6 ms. / P1 waits from 3 to 8 (5ms) and from 11 to 12 (1ms). Total = 6ms.

- (c) İki periyodik gerçek zamanlı (real-time) görev içeren bir sistem düşünelim: (*English: Consider a real-time system running two periodic tasks.*)

- Görev 1 (τ_1): İşlem Süresi $C_1 = 2$ ms, Periyot $P_1 = 5$ ms.
- Görev 2 (τ_2): İşlem Süresi $C_2 = 3$ ms, Periyot $P_2 = 10$ ms.

(*English: Task 1 (τ_1): Execution Time $C_1 = 2$ ms, Period $P_1 = 5$ ms. Task 2 (τ_2): Execution Time $C_2 = 3$ ms, Period $P_2 = 10$ ms.*)

- (a) Bu sistemin toplam CPU kullanım oranını (U) hesaplayınız. (*English: Compute the total CPU utilization (U) of this system.*)

Answer (Cevap): _____ **0.7** _____

Solution: $U = \frac{2}{5} + \frac{3}{10} = 0.4 + 0.3 = 0.7$.

- (b) **Rate Monotonic Scheduling (RMS)** statik öncelik algoritmasına göre hangi görev daha yüksek önceliğe sahiptir? (*English: Under the **Rate Monotonic Scheduling (RMS)** static-priority algorithm, which task is assigned the higher priority?*)

Final Answer (Son Cevap): Görev 1 / Task 1

Solution: $P_1(5ms) < P_2(10ms)$ olduğu için Görev 1 daha yüksek önceliğe sahiptir. / Since $P_1 < P_2$, Task 1 has higher priority.

- (c) Bu iki görev kümesi **Earliest Deadline First (EDF)** dinamik öncelik algoritması altında kaçırmadan başarıyla planlanabilir mi? (**Yes** veya **No** yazınız). (*English: Can this task set be successfully scheduled without missing deadlines under the **Earliest Deadline First (EDF)** dynamic algorithm? (Write **Yes** or **No**).*)

Answer (Cevap): _____ **Yes** _____

Part III: Memory Virtualization & Locality Code Trace

20

3. (a) 32-bitlik bir mimaride 2-seviyeli hiyerarşik sayfalama kullanılmaktadır. Sayfa boyutu **4 KB** (2^{12} bytes) ve her bir Sayfa Tablosu Elemanı (PTE) **4 byte** uzunluğundadır. Tek bir sayfa tablosunun bir sayfa çerçevesine tam sığabilmesi için 32-bitlik sanal adresin nasıl bölünmesi gerektiğini yazınız. (*English: A 32-bit architecture uses a 2-level hierarchical paging scheme. The page size is **4 KB**, and each Page Table Entry (PTE) takes **4 bytes**. Calculate the exact bit split for the 32-bit virtual address.*)

- Dış Sayfa Tablosu indeksi için bit sayısı / Number of bits for Outer Page Table index:
10 bits
- İç Sayfa Tablosu indeksi için bit sayısı / Number of bits for Inner Page Table index:
10 bits
- Sayfa içi sapma (offset) için bit sayısı / Number of bits for Page Offset:
12 bits

- (b) Bir programda `int A[128][128];` şeklinde bir matris tanımlanmıştır. Bir `int` verisi **4 byte** yer kaplamaktadır. Sistemde **sayfa boyutu 512 byte**'tır (128 eleman) ve programa ayrılmış yalnızca 1 adet boş fiziksel bellek çerçevesi vardır. Matris bellekte satır-bazlı (row-major) sıralı tutulmaktadır. Aşağıdaki iki farklı kod fragmanı çalıştırıldığında oluşacak toplam sayfa hatası (page fault) sayısını bulunuz (Döngü değişkenlerini ihmal ediniz). (*English: An array `int A[128][128];` is defined where an `int` is **4 bytes**. The system has a page size of 512 bytes (holds exactly 128 integers) and the process is allocated only 1 physical memory frame. The array is stored in row-major order. Calculate the total number of page faults.*)

- (a) **Fragment 1 (Satır bazlı gezinme / Row-based traversal):**

```
for (int i = 0; i < 128; i++) {
    for (int j = 0; j < 128; j++) {
        A[i][j] = 0;
    }
}
```

Fragment 1 için toplam sayfa hatası / Total Page Faults for Fragment 1:
128

- (b) **Fragment 2 (Sütun bazlı gezinme / Column-based traversal):**

```
for (int j = 0; j < 128; j++) {
    for (int i = 0; i < 128; i++) {
        A[i][j] = 0;
    }
}
```

Fragment 2 için toplam sayfa hatası / Total Page Faults for Fragment 2:
16384

- (c) Toplam **3 adet boş fiziksel çerçeveye** sahip bir sisteme sırasıyla şu sayfa istekleri gelmektedir: 2, 3, 2, 1, 5, 2, 4.

Second-Chance (Clock) Algoritmasını kullanarak oluşacak toplam sayfa hatası sayısını bulunuz. (*English: A process makes the following sequential memory page references to a system with 3 empty physical frames: 2, 3, 2, 1, 5, 2, 4. Using the **Second-Chance (Clock) Algorithm**, calculate the total number of page faults.*)

Toplam Sayfa Hatası / Total Page Faults:

5

Part IV: File Systems & Disk I/O

20

4. (a) Bir C programı içerisinde şu sistem çağrıları yapılmıştır: (*English: The following system calls are made in a C program.*)

```
int fd1 = open("log.txt", O_RDONLY);
int fd2 = dup(fd1); //duplicates fd1/fd1nin kopyasi olusturuldu
```

Boşlukları **Shared** veya **Unique** kelimeleriyle doldurunuz: (*English: Fill in the blanks with **Shared** or **Unique** based on the snippet above.*)

- (a) Sistem genelinde Açık Dosya Tablosu girişleri / System-wide Open File Table entries:

Shared

(b) Sistem genelinde Inode Tablosu girişleri / System-wide Inode Table entries:

Shared

- (c) Disk kafası **başlangıçta 63** konumundadır. Verilen algoritmaların istekleri hangi sıra ile işleyeceğini boşluklara yazınız. (*English: The disk head is initially at position 63. Write the exact service order for each given algorithm.*)

İstek kuyruğu (Requests queue): 60, 71, 147, 17, 106, 80, 148, 36, 73, 102

- (a) **FCFS (First-Come, First-Served):**

60, 71, 147, 17, 106, 80, 148, 36, 73, 102

- (b) **SSTF / SPTF (Shortest Seek Time First):**

60, 71, 73, 80, 102, 106, 147, 148, 36, 17

Part V: Access Control & Security Layer

20

5. (a) Bir sistemde üç farklı dosya (`syllabus.txt`, `exam.pdf`, `submission.zip`) ve üç farklı rol (**Prof**, **TA**, **Stu**) bulunmaktadır. (*English: There are three files and three distinct roles (**Prof**, **TA**, **Stu**).*)

- (a) Aşağıdaki kurallara göre **Access Control Matrix** tablosunu doldurunuz (**r**, **w**, **x** veya **-** kullanarak): (*English: Fill in the **Access Control Matrix** based on these rules (use **r**, **w**, **x** or **-**).*)

- **Prof** tüm dosyaları okuyabilir, yazabilir ve çalıştırabilir. / can read, write, and execute all files.
- **TA** tüm dosyaları okuyabilir, ancak yalnızca `submission.zip`'e yazabilir. Çalıştırma yetkisi yok. / can read all files, but can only write to `submission.zip`. No execute rights.
- **Stu** yalnızca `syllabus.txt`'yi okuyabilir. `submission.zip`'i okuyabilir, yazabilir ve çalıştırabilir. `exam.pdf` üzerinde hiçbir izni yoktur. / can only read `syllabus.txt`. Can read, write, and execute `submission.zip`. Has no permissions on `exam.pdf`.

Role / User	syllabus.txt	exam.pdf	submission.zip
Prof	<u>rwX</u>	<u>rwX</u>	<u>rwX</u>
TA	<u>r</u>	<u>r</u>	<u>rw</u>
Stu	<u>r</u>	<u>-</u>	<u>rwX</u>

- (b) `submission.zip` dosyasının **Access Control List (ACL)** ifadesini yazınız. (*English: Write out the explicit **Access Control List (ACL)** for the file `submission.zip`.*)

submission.zip için ACL / ACL for submission.zip:

Prof: rwX, TA: rw, Stu: rwX

- (c) Aşağıda verilen gerçek dünya senaryolarının hangi erişim kontrol modeline (**DAC** veya **MAC**) ait olduğunu boşluklara yazınız. (*English: Identify whether each real-world security policy matches **DAC** (Discretionary) or **MAC** (Mandatory).*)

- (a) Alice, bir Linux iş istasyonunda bir metin belgesi oluşturur ve Bob'un okumasını tamamen engellemek için `chmod 600 report.txt` komutunu çalıştırır. Dosya sahibi erişim kurallarını belirlediğinden bu politika: (*English: Alice creates a text document on a Linux workstation and runs `chmod 600 report.txt` to completely block Bob from reading it. Since the file owner decides the access rules, this represents an example for the policy:)*

DAC

- (b) Bir hava kuvvetleri üssü işletim sistemi, Albay Smith'e "Top Secret" güvenlik açıklama etiketi ve taktik bir dosyaya "Secret" etiketi atar. Smith bu sistem geneli yönetsel etiketleri değiştiremez. Bu politika: (*English: An air force base operating system assigns a "Top Secret" security clearance label to Colonel Smith and a "Secret" label to a tactical file. Smith cannot alter these system-wide administrative labels. This represents an example for the policy:*)

MAC

(c) Bellek ve yetkilendirme güvenliğiyle ilgili aşağıdaki senaryoları inceleyerek soruları cevaplandırınız. *(English: Examine the following system security scenarios and answer the corresponding questions.)*

- (a) Bir program sınır kontrolü yapmayan `strcpy()` fonksiyonunu kullanmaktadır. Girdi sınırları taşırılıp fonksiyonun dönüş adresi (return address) manipüle edilirse hangi kritik güvenlik açığı (**vulnerability**) oluşur? *(English: A program utilizes `strcpy()` without bound tracking. If a malicious input overflows memory boundaries to overwrite the return address, what critical security vulnerability is triggered?)*

Answer (Cevap): Buffer Overflow (Tampon Taşması)

- (b) Linux tabanlı sistemlerde normal bir kullanıcının root yetkilerine ihtiyaç duyan şifre değiştirme programını (`passwd`) çalıştırabilmesini sağlayan özel dosya izni biti nedir? *(English: In Linux systems, what specific executable file permission bit allows standard users to run programs like `passwd` with temporary root privileges?)*

Answer (Cevap): setuid

- (c) İşletim sistemlerinde Buffer Overflow saldırılarını veya bu açıkların sömürülmesini engellemek için kullanılan yöntemlerden/mekanizmalardan birinin adını yazınız. *(English: Name one operating system hardware or software protection mechanism used to detect, mitigate, or prevent code injection via Buffer Overflow vulnerabilities.)*

Answer (Cevap): Stack Canaries

Solution: Stack Canaries (canary), NX bit (No-Execute), DEP (Data Execution Prevention), ASLR (Address Space Layout Randomization) gibi cevaplar kabul edilir. / Answers such as Stack Canaries, NX bit, DEP, or ASLR are accepted.

End of Exam.